

INSTITUTO TECNOLÓGICO DE COSTA RICA
Carrera: Ingeniería en Computadores · Introducción a la Programación

Documentación del Proyecto

Atributos de Análisis de Problema y Herramientas de Ingeniería

Esteban Sánchez Ledezma — Carné: 2026108570
Dominick Robles Samudio — Carné: 2026093878

Profesor(a): Jeff Schmidt Peralta · Grupo: 2
Fecha: 28/06/2026 · I Semestre 2026

1. Atributo de Análisis de Problema

1.1 Identificación del problema complejo de ingeniería

El problema central de este proyecto consiste en diseñar e implementar un sistema de juego de estrategia por turnos para dos jugadores en tiempo real compartido, lo que implica resolver simultáneamente varios sub-problemas de ingeniería de software de naturaleza compleja:

- Gestión de estados concurrentes: el sistema debe mantener y transicionar de forma correcta entre cuatro fases de ejecución (construcción, ataque, combate y resultado) sin perder coherencia en los datos de cada jugador.
- Modelado espacial y algorítmico: el mapa de 15×15 casillas requiere un modelo de datos eficiente (dict con claves (fila, columna)) para representar posiciones, calcular zonas de defensa mediante conjuntos (set), y ejecutar un algoritmo de movimiento con detección de obstáculos y alcance en cada turno de combate.
- Diseño orientado a objetos con polimorfismo: el sistema debe manejar múltiples tipos de entidades (torres, unidades, muros, base) con comportamientos distintos pero interfaz común, lo que se resuelve mediante clases abstractas y herencia.
- Persistencia y seguridad de datos: el sistema de cuentas de usuario exige almacenamiento permanente entre ejecuciones (JSON), validación de entrada y protección de contraseñas mediante hash criptográfico (SHA-256).
- Sincronización de interfaz gráfica y lógica: la fase de combate debe resolverse automáticamente turno a turno con intervalos de tiempo controlados, sin bloquear la interfaz de Tkinter.

1.2 Análisis del contexto, variables y factores – Desarrollo sostenible

El proyecto se desarrolla en el contexto de un curso universitario de introducción a la programación, donde los integrantes trabajan de forma remota desde ubicaciones geográficamente distantes. Esto introduce variables adicionales que van más allá del código:

- Coordinación asíncrona: la comunicación se realizó principalmente mediante herramientas digitales (WhatsApp, Telegram, Discord y Zoom), lo que implicó planificar el trabajo de forma modular para que cada integrante pudiera avanzar de forma independiente.
- Accesibilidad del software: el juego se desarrolló en Python con Tkinter, tecnologías de código abierto y gratuitas, lo que garantiza que cualquier persona con una computadora básica pueda ejecutarlo sin costo adicional. Esto se alinea con el principio de acceso equitativo a la tecnología.
- Software educativo: el proyecto tiene valor pedagógico directo – como herramienta de aprendizaje de programación orientada a objetos, estructuras de datos y diseño de interfaces – lo que contribuye al desarrollo de competencias técnicas en el equipo.
- Uso responsable de inteligencia artificial: se utilizó IA como herramienta de apoyo para organización, revisión de lógica y generación de sprites, sin delegar el desarrollo del código. Esto

refleja un uso ético y consciente de la IA como herramienta que potencia la productividad sin reemplazar el aprendizaje.

- Eficiencia de recursos computacionales: el sistema utiliza únicamente librerías de la biblioteca estándar de Python (excepto Pygame), evitando dependencias pesadas y reduciendo el consumo de recursos del sistema.

1.3 Plan de solución propuesto

La solución se estructuró en tres capas claramente separadas:

- Capa de presentación (Pantallas): cada pantalla del juego es una clase independiente con su propio ciclo de vida (mostrar, interactuar, navegar). La navegación se implementa mediante callbacks, lo que evita dependencias cíclicas entre pantallas y permite reutilizar la misma ventana Tkinter.
- Capa de lógica de juego (Mapa, Motor, Entidades): el MapaJuego encapsula el estado del tablero mediante un diccionario de ocupantes. El MotorCombate ejecuta los turnos de forma desacoplada del mapa. Las entidades (Torres, Unidades) implementan sus propias habilidades especiales mediante el patrón Template Method (método abstracto activar_habilidad()).
- Capa de datos (GestorUsuarios, Billetera, EconomiaController): el sistema de cuentas persiste en un archivo JSON con contraseñas hasheadas. La economía del juego se centraliza en EconomiaController, que coordina las dos billeteras (defensor y atacante) y aplica recompensas y bonos según los eventos del combate.

1.4 Evaluación de la solución – Ventajas y desventajas

Dimensión	Ventajas	Desventajas
Técnica	Diseño OOP sólido con ABC. Separación clara de capas. Código modular y extensible.	Todo el código en un solo archivo main.py. Dificultad para escalar sin refactorización.
Económica	Cero costo de desarrollo: herramientas gratuitas y de código abierto.	Requiere Python instalado. No es una aplicación distribuable sin empaquetado adicional.
Ambiental	Aplicación de escritorio liviana. Bajo consumo energético. Sin servidores ni infraestructura cloud.	Ejecución local; sin posibilidad de juego en red sin rediseño arquitectónico.
Social	Accesible para cualquier persona con Python. Fomenta el juego cara a cara en la misma computadora.	Requiere que ambos jugadores compartan la misma computadora (no hay modo multijugador en red).

2. Atributo de Herramientas de Ingeniería

2.1 Técnicas, recursos y herramientas utilizadas

A continuación se describen las técnicas de ingeniería y las herramientas empleadas durante el desarrollo del proyecto:

Categoría	Herramienta / Recurso	Uso en el proyecto
Lenguaje	Python 3.14	Lenguaje principal del juego: lógica, clases, control de flujo.
Interfaz gráfica	Tkinter	Construcción de todas las pantallas, canvas, botones y widgets.
Audio	Pygame	Reproducción de música de fondo y control de pausa/reanudación.
Persistencia	JSON	Almacenamiento y lectura del archivo usuarios.json.
Seguridad	hashlib	Hasheado de contraseñas antes de guardarlas en el archivo.
Control de versiones	Git + GitHub	Gestión de ramas, commits y respaldo del código fuente.
Editor de código	Visual Studio Code	Desarrollo, depuración y navegación del código.
Documentación	Microsoft Word	Redacción de la documentación técnica y manuales del proyecto.
Hoja de cálculo	Google Sheets	Planificación de costos, estadísticas de balance económico del juego.
Almacenamiento	Google Drive	Subida y compartición de videos de evidencia del desarrollo.
Comunicación	WhatsApp / Telegram / Discord	Coordinación diaria entre integrantes que viven en lugares distintos.
Videollamadas	Zoom / Google Meet	Sesiones de trabajo conjunto, revisiones de código y diseño.
Asistencia IA	Claude, Chat GPT y Gemini	Apoyo en organización, revisión de lógica, búsqueda y corrección en redacciones. No reemplazó el desarrollo; se usó como herramienta de apoyo.
Diagramas	draw.io y Canva	Generación de diagramas de clases para la documentación.

Las técnicas de ingeniería de software aplicadas incluyen:

- Programación orientada a objetos (POO): uso de clases abstractas (ABC), herencia, encapsulamiento y polimorfismo para modelar las entidades del juego.

- Patrón Template Method: definido mediante @abstractmethod en las clases Torre y Unidad, obligando a cada subclase a implementar su propia activar_habilidad().
- Patrón Callback / Observer implícito: la navegación entre pantallas se implementa pasando funciones (callbacks) como parámetros, desacoplando las pantallas entre sí.
- Hash criptográfico: las contraseñas se procesan con SHA-256 antes de ser almacenadas, aplicando principios básicos de seguridad en software.

2.2 Diagramas de clases

Se presentan a continuación tres diagramas de clases que representan los tres subsistemas principales del proyecto. Los diagramas se generaron a partir del código fuente de main.py.

Diagrama 1 – Sistema de pantallas y navegación

Muestra la clase Utilidades (servicios compartidos) y todas las clases de pantalla con sus métodos principales. Las flechas punteadas indican relaciones de navegación mediante callbacks.

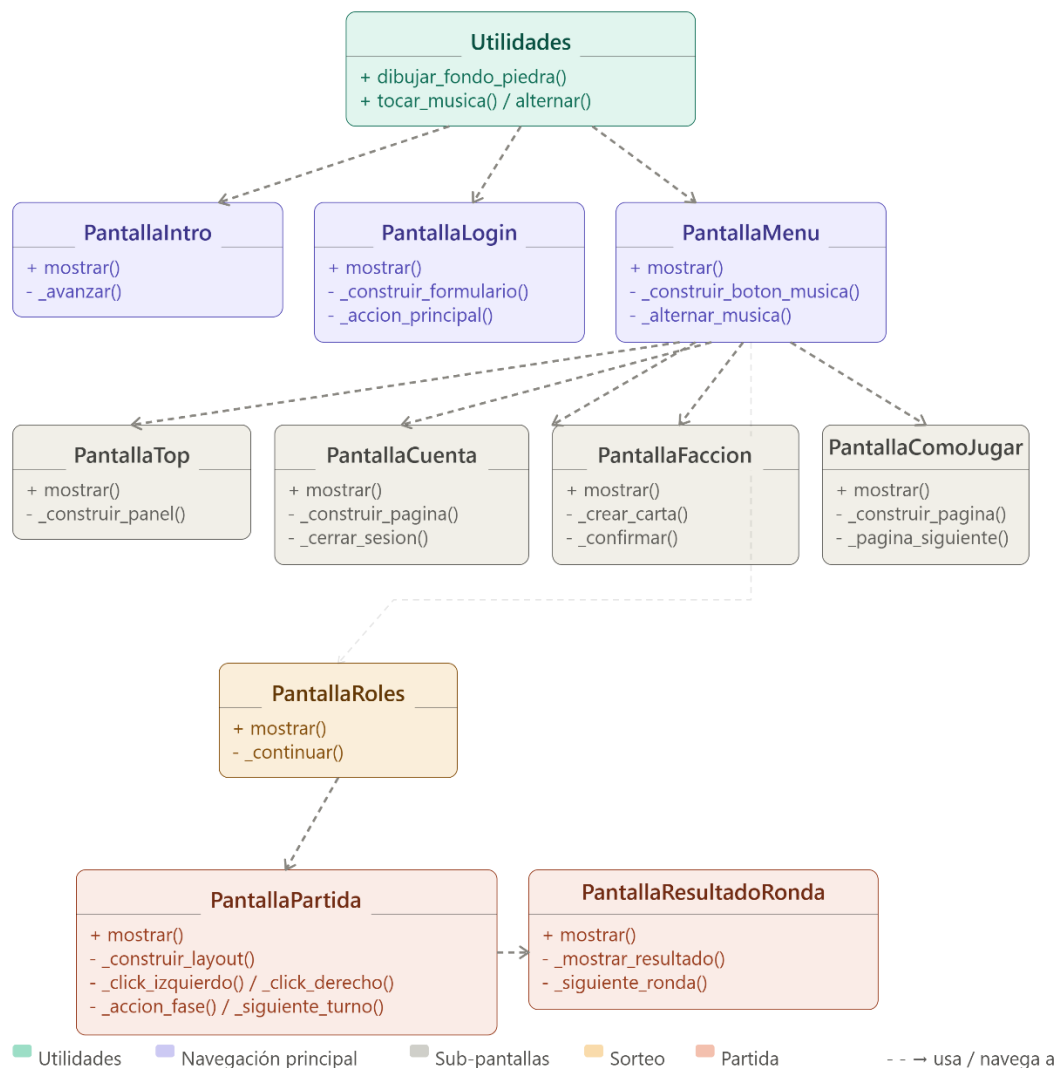


Diagrama 2 – Torres, Unidades, Base y Motor de Combate

Muestra la jerarquía de herencia de Torre (ABC) → TorreVigia, TorreCanon, TorreSanadora y Unidad (ABC) → Soldado, Tanque, UnidadRapida. Incluye Muro, BaseCentral, MapaJuego y MotorCombate.

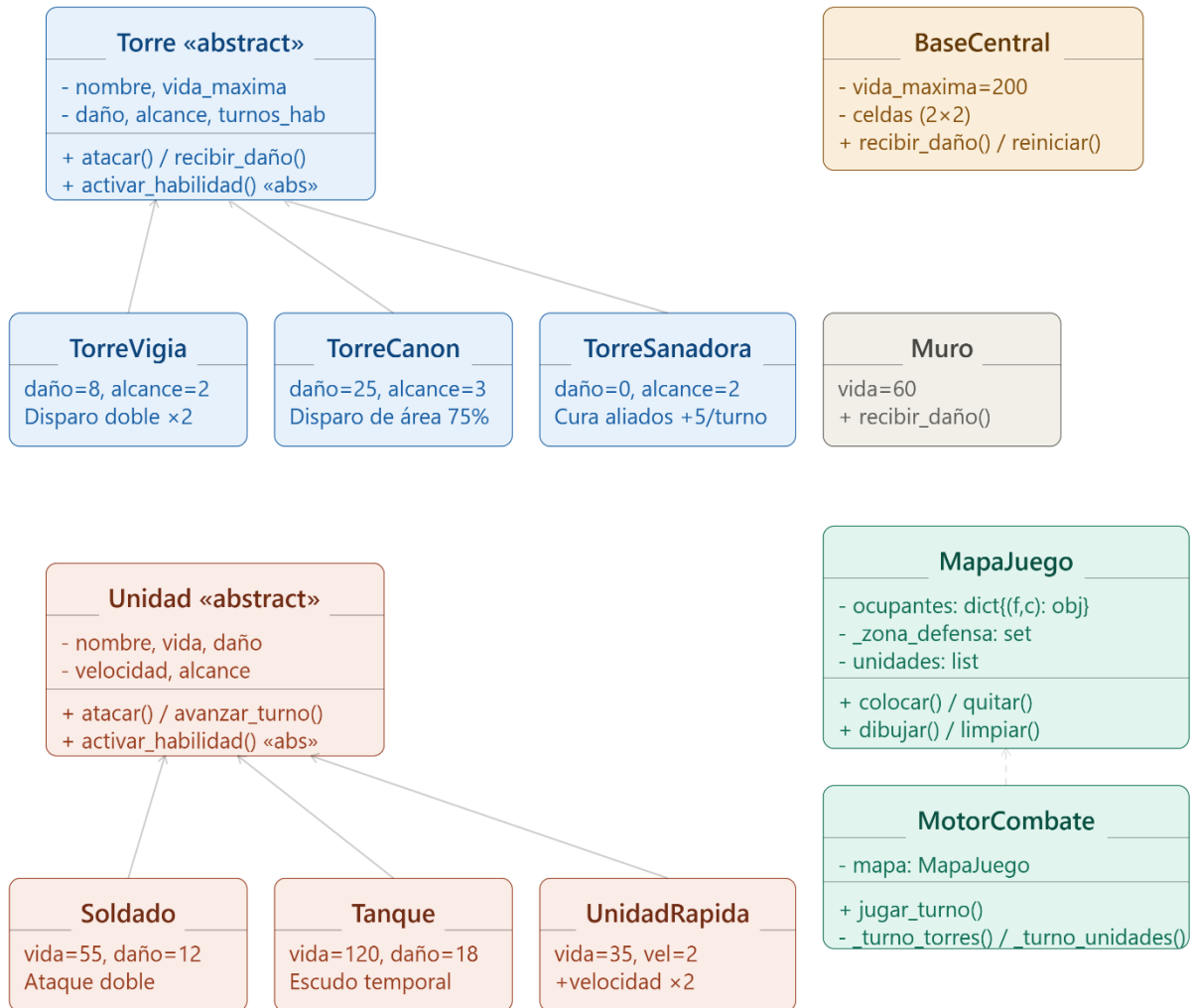
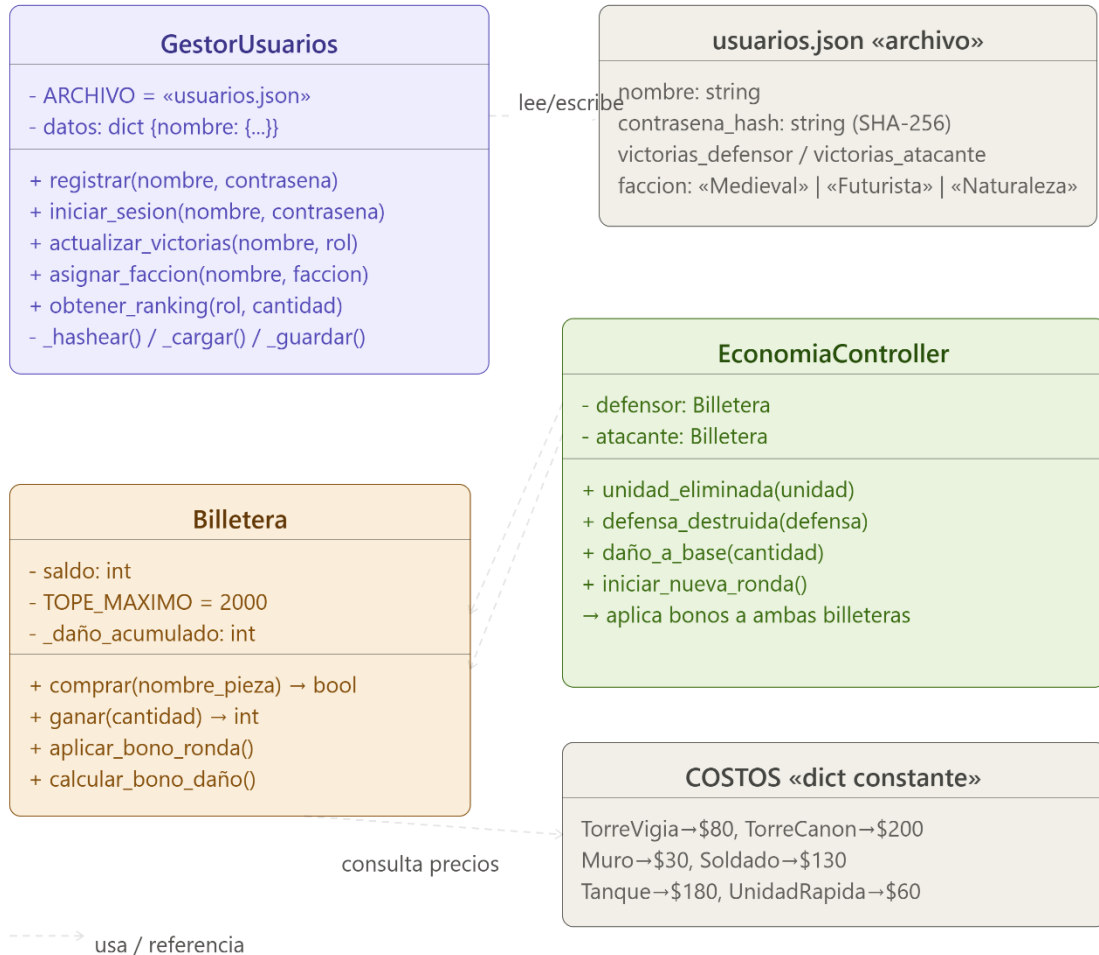


Diagrama 3 – Sistema de usuarios, cuentas y economía

Muestra *GestorUsuarios* (acceso a *usuarios.json*), *Billetera* (saldo y bonos de cada jugador) y *EconomiaController* (coordinador de recompensas durante el combate), junto al diccionario constante *COSTOS*.



2.3 Aplicación de las herramientas durante el desarrollo

El uso de las herramientas no fue estático; evolucionó a lo largo del proyecto:

- **Python + Tkinter:** se comenzó con un prototipo de mapa básico con turnos manuales (un botón para avanzar el combate). Posteriormente se implementó el sistema de fases automáticas con `ventana.after()`, lo que requirió comprender el bucle de eventos de Tkinter para no bloquear la interfaz.
- **Git + GitHub:** se usó desde el primer día para registrar el avance incremental. Cada funcionalidad nueva se desarrolló y probó antes de hacer `commit`, lo que permitió revertir cambios problemáticos en varias ocasiones.
- **Discord + Zoom:** al vivir en lugares distintos, algunas sesiones de programación se realizaron por videollamada, compartiendo pantalla para resolver bugs complejos en tiempo real.

- Google Sheets: se usó para balancear la economía del juego – simular partidas con distintos costos y recompensas hasta encontrar valores que generaran partidas competitivas.

2.4 Adaptaciones y refinamientos durante el desarrollo

El proyecto experimentó cambios significativos frente al diseño inicial:

- El combate pasó de manual (botón por turno) a automático (`ventana.after()` con intervalos de 1500ms), lo que mejoró notablemente la experiencia de juego pero requirió rediseñar el flujo de control para evitar condiciones de carrera con los eventos de Tkinter.
- El mapa original era de 10×10 casillas. Al probar el juego con más unidades, se evidenció que el espacio era insuficiente, por lo que se amplió a 15×15 y se ajustó el layout de la ventana en consecuencia (log movido a columna derecha para no aumentar la altura de la ventana).
- El sistema de skins se diseñó inicialmente solo para torres y muros. Al agregar los sprites de las tropas y la base central, fue necesario refactorizar `_cargar_imagen()` para manejar dos facciones distintas (defensor y atacante) y un caché de imágenes compartido.
- La navegación entre pantallas comenzó con referencias directas entre clases. Al detectar acoplamiento fuerte, se refactorizó completamente a callbacks pasados como parámetros en el constructor de cada pantalla, lo que permitió que el flujo de la aplicación se controlara desde las funciones globales `ir_a_*`() al final del archivo.